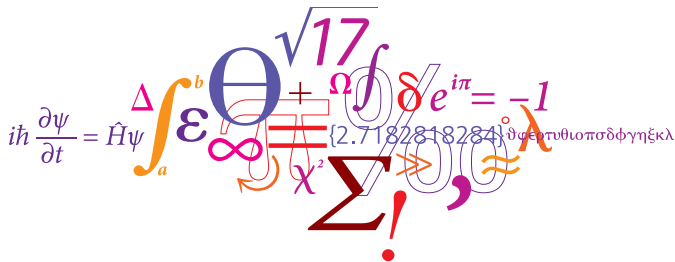


Final Project - Quantum teleportation

Oskar Brunn Fugmann (s224073)

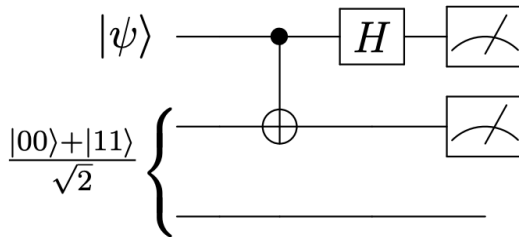


Introduction

- Quantum teleportation of qubit
- Entanglement swapping
- Quantum teleportation with continuous variable

Quantum teleportation

- Quantum teleportation circuit
- Do correction to Bob state
 - If 1'st bit is 1 $\Rightarrow$ Z
 - If 2'nd bit is 1 $\Rightarrow$ X



My Implementation

```
# define the quantum circuit for quantum teleportation
q = QubitCircuit(3, num_cbits=2, reverse_states=False)
# add the cnot gate on qubit 1 with control on the 0'th qubit
q.add_gate("CNOT", controls=[0], targets=[1])
# add the Hadamard gate on qubit 0
q.add_gate("H", targets=[0])
# Do alice measurement on qubit 0 and 1 and save the measurement as classical bits
q.add_measurement("M0", targets=[0], classical_store=0)
q.add_measurement("M1", targets=[1], classical_store=1)
# Do the correction depending on Alices measurements
q.add_gate("X", targets=[2], classical_controls=[1])
q.add_gate("Z", targets=[2], classical_controls=[0])
```

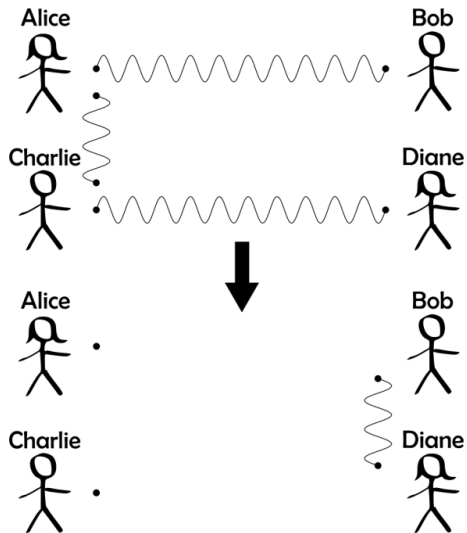
Creating noise

- Loss
- Random bit-flip
- Gates are imperfect

```
# The error is set to a random value from [-epsilon,epsilon]
error = rng.uniform(epsilon, epsilon)
# define the quantum circuit for quantum teleportation
q = QubitCircuit(3, num_cbits=2, reverse_states=False)
# add the cnot gate on qubit 1 with control on the 0'th qubit
# The RX gate is to apply a tiny error
q.add_gate("RX", targets=[0], arg_value=error)
q.add_gate("CNOT", controls=[0], targets=[1])
# add the Hadamard gate on qubit 0
# The RX gate is to apply a tiny error
q.add_gate("RX", targets=[0], arg_value=error)
q.add_gate("H", targets=[0])
# Do alice measurement on qubit 0 and 1 and save the measurement as classical bits
q.add_measurement("M0", targets=[0], classical_store=0)
q.add_measurement("M1", targets=[1], classical_store=1)
# Do the correction depending on Alice's measurements
# here the rotation is a tiny bit larger or smaller than expected creating error
q.add_gate("RX", targets=[2], classical_controls=[1], arg_value=error + np.pi)
q.add_gate("RZ", targets=[2], classical_controls=[0], arg_value=error + np.pi)
```

Entanglement swapping

- Teleports bell state
- Use twice the quantum teleport circuit



My Implementation

```
e = QubitCircuit(6, num_cbits=4, reverse_states=False)
# Creates a circuit that teleports Alice part of the state we want to teleport to charlie
# and teleports Bobs part to Diane
e.add_gate("CNOT", controls=[0], targets=[2])
e.add_gate("CNOT", controls=[1], targets=[4])
e.add_gate("H", targets=[0])
e.add_gate("H", targets=[1])
# measuse the qubits that Alice and Bob has access to.
e.add_measurement("M0", targets=[0], classical_store=0)
e.add_measurement("M1", targets=[2], classical_store=1)
e.add_measurement("M0", targets=[1], classical_store=2)
e.add_measurement("M1", targets=[4], classical_store=3)
# applies a correction such that Chalie and Diane shares an entangled state.
e.add_gate("X", targets=[3], classical_controls=[1])
e.add_gate("Z", targets=[3], classical_controls=[0])
e.add_gate("X", targets=[5], classical_controls=[3])
e.add_gate("Z", targets=[5], classical_controls=[2])
```

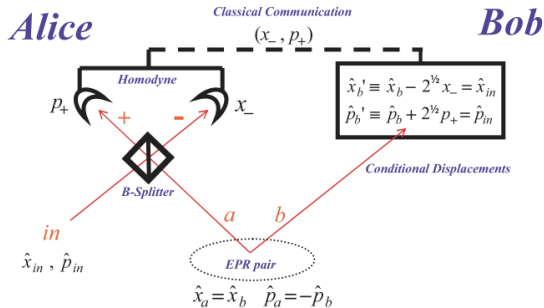
How do i return kets

- qutip_qip.circuit
- measurement does not destroy the state

```
#Find the dimension of the hilbertspace ie. the amount of qubits in the hilbert space.
n = len(state.dims[0])
#Creates a list of all possible states in this hilbertspace
bits = [format(i, f"0{n}b") for i in range(2 ** n)]
#Creates a list which will contain the where each element in the state belongs
# to in the output. So for a qubit bit will be two long so |0> and |1>.
# And for a bell state bit will be 4 long so |00>, |01>, |10> and |11>
bit = []
for b in bits:
    value = ''.join(b[-(p + 1)] for p in positions)
    bit.append(int(value, 2))
#take the state that have been teleported
vector = state.full().flatten()
#Take the amplitude of each element in the statevector and appends it to the right place
# to create the amplitude of the output state and then returns that list.
kets = [np.array([]) for _ in range(2*len(positions))]
for i, amplitude in enumerate(vector):
    kets[bit[i]] = np.append(kets[bit[i]], amplitude)
sums = [np.sum(array) for array in kets]
```


Quantum teleportation using continuous variable

- Quantum teleportation circuit
- EPR $\Rightarrow$ 2-mode squeezing
- Homodyne detection to displacement



My Implementation

```
#define state to teleport
psi_input = qu.coherent(N, alpha)
#define annihilation operators
a = qu.destroy(N)
I = qu.qeye(N)
a0 = qu.tensor(a, I, I) # input
a1 = qu.tensor(I, a, I) # Alice
a2 = qu.tensor(I, I, a) # Bob
#Define 2 mode squeezing
S2 = (r * (a1 * a2 - a1.dag() * a2.dag())).expm()
#define full system state of the state we want to teleport and an EPR state.
vac = qu.tensor(psi_input, qu.basis(N, 0), qu.basis(N, 0))
state = S2 * vac
#Define the beamsplitter acting on input and Alice
theta = np.pi / 4
U_bs = (theta * (a0.dag() * a1 - a0 * a1.dag())).expm()
#apply beamsplitter on the state
BsState = U_bs * state
```

Quantum teleportation using continuous variable Gaussian picture

$$\hat{\mathbf{r}} = \begin{pmatrix} \hat{x}_1 \\ \hat{p}_1 \\ \vdots \\ \hat{x}_n \\ \hat{p}_n \end{pmatrix}$$

$$\sigma \rightarrow F\sigma F^T, \quad \bar{r} \rightarrow F\bar{r} + d$$

```
# create the system
# define the quadrature values as a vector
x0 = np.sqrt(2) * np.real(alpha)
p0 = np.sqrt(2) * np.imag(alpha)
quad = np.array([x0, p0, 0.0, 0.0, 0.0, 0.0])
# define covariance matrix:
sigma = 0.5 * np.eye(6)
# Apply the two operations on the vector
# of quadratures and the covariance matrix
quad = F_TMS @ quad
quad = F_BS @ quad
sigma = F_TMS @ sigma @ F_TMS.T
sigma = F_BS @ sigma @ F_BS.T
```